数据结构

1 线段树

1.1 普通线段树

```
class SegmentTree
{
private:
    int n,*res,*tag;
    void pushup(int k){res[k]=res[k<<1]+res[k<<1|1];}
    void pushdown(int k,int l,int r)
    {
        int mid=l+r>>1;
        res[k<<1]+=(mid-l+1)*tag[k],res[k<<1|1]+=(r-mid)*tag[k];
        tag[k<<1]+=tag[k],tag[k<<1|1]+=tag[k];
        tag[k]=0;
    }
    void update(int k,int l,int r,int x,int y,int z)
    {
        if(y<l||x>r) return;
        if(x<=l&&r<=y)
        {
            res[k]+=(r-l+1)*z;
            tag[k]+=z;
            return ;
        }
        pushdown(k,l,r);
        int mid=l+r>>1;
        update(k<<1,l,mid,x,y,z);
        update(k<<1|1,mid+1,r,x,y,z);
        pushup(k);
    }
    int query(int k,int l,int r,int x,int y)
    {
        if(y<l||x>r) return 0;
        if(x<=l&&r<=y) return res[k];
        pushdown(k,l,r);
        int mid=l+r>>1;
        return query(k<<1,l,mid,x,y)+query(k<<1|1,mid+1,r,x,y);
    }
public:
    SegmentTree(int size)
    {
        n=size;
        res=new int[(n<<2)+5]();
        tag=new int[(n<<2)+5]();
    }
    ~SegmentTree()
    {
        delete [] res;
        delete [] tag;
    }
```

```
    void update(int l,int r,int w){update(1,1,n,l,r,w);}
    int query(int l,int r){return query(1,1,n,l,r);}
};
```

1.2 可持久化线段树

静态区间第K大

权值线段树维护前缀和，即查询在`[1,i]`区间中有多少元素的值在`[l,r]`之间

```
#include<bits/stdc++.h>
using namespace std;
const int N=2e5+5;
int n,m,q,a[N],b[N];
class SegmentTree
{
#define ls(x) ch[x][0]
#define rs(x) ch[x][1]
private:
    int sum[N<<5],ch[N<<5][2],tot,t[N];
    void update(int pre,int &now,int l,int r,int w)
    {
        if(w<l||w>r) return ;
        now=++tot;
        ls(now)=ls(pre),rs(now)=rs(pre),sum[now]=sum[pre]+1;
        if(l==r) return ;
        int mid=l+r>>1;
        update(ls(pre),ls(now),l,mid,w);
        update(rs(pre),rs(now),mid+1,r,w);
    }
    int query(int u,int v,int l,int r,int k)
    {
        if(l==r) return l;
        int w=sum[ls(v)]-sum[ls(u)],mid=l+r>>1;
        return w>=k?query(ls(u),ls(v),l,mid,k):query(rs(u),rs(v),mid+1,r,k-w);
    }
public:
    void insert(int i){update(t[i-1],t[i],1,m,lower_bound(b+1,b+1+m,a[i])-b);}
    int ask(int l,int r,int k){return b[query(t[l-1],t[r],1,m,k)];}
#undef ls
#undef rs
}t;
int main()
{
    scanf("%d%d",&n,&q);
    for(int i=1;i<=n;i++)
        scanf("%d",&a[i]),b[i]=a[i];
    sort(b+1,b+1+n);
    m=unique(b+1,b+1+n)-b-1;
    for(int i=1;i<=n;i++)
        t.insert(i);
    while(q--)
    {
        int l,r,k;
```

```
        scanf("%d%d%d",&l,&r,&k);
        printf("%d\n",t.ask(l,r,k));
    }
    return 0;
}
```

2 平衡树

2.1 Splay

```
class Splay
{
#define ls(x) ch[0][x]
#define rs(x) ch[1][x]
private:
    int rt,tot;
    queue<int>q;
    vector<int>ch[2],f,s,v,cnt;
    bool chk(int x){return rs(f[x])==x;}
    void pushup(int k){s[k]=s[ls(k)]+s[rs(k)]+cnt[k];}
    void init()
    {
        ch[0].push_back(0);
        ch[1].push_back(0);
        f.push_back(0);
        s.push_back(0);
        v.push_back(0);
        cnt.push_back(0);
    }
    void rotate(int x)
    {
        int y=f[x],z=f[y],k=chk(x);
        ch[k][y]=ch[k^1][x],f[ch[k^1][x]]=y;
        ch[chk(y)][z]=x,f[x]=z;
        ch[k^1][x]=y,f[y]=x;
        pushup(y),pushup(x);
    }
    void splay(int x,int k=0)
    {
        while(f[x]!=k)
        {
            int y=f[x],z=f[y];
            if(z!=k)
            {
                if(chk(x)==chk(y)) rotate(y);
                else rotate(x);
            }
            rotate(x);
        }
        if(!k) rt=x;
    }
    void find(int x)
    {
        int cur=rt;
```

```
        while(ch[x>v[cur]][cur]&&x!=v[cur])
            cur=ch[x>v[cur]][cur];
        splay(cur);
    }
    int kth_t(int k)
    {
        int cur=rt;
        while(1)
        {
            if(ls(cur)&&k<=s[ls(cur)])
                cur=ls(cur);
            else if(k>s[ls(cur)]+cnt[cur])
            {
                k-=s[ls(cur)]+cnt[cur];
                cur=rs(cur);
            }
            else return cur;
        }
    }
    int pre_t(int x)
    {
        find(x);
        if(v[rt]<x) return rt;
        int cur=ls(rt);
        while(rs(cur)) cur=rs(cur);
        return cur;
    }
    int suc_t(int x)
    {
        find(x);
        if(v[rt]>x) return rt;
        int cur=rs(rt);
        while(ls(cur)) cur=ls(cur);
        return cur;
    }
public:
    Splay()
    {
        rt=0,tot=0;
        init();
        insert(0x3f3f3f3f);
        insert(0xcfcfcfcf);
    }
    void insert(int x)
    {
        int cur=rt,fa=0;
        while(cur&&v[cur]!=x)
        {
            fa=cur;
            cur=ch[x>v[cur]][cur];
        }
        if(cur) cnt[cur]++;
        else
        {
            if(!q.empty()) cur=q.front(),q.pop();
```

```
            else cur=++tot,init();
            if(fa) ch[x>v[fa]][fa]=cur;
            ls(cur)=rs(cur)=0;
            f[cur]=fa,v[cur]=x;
            cnt[cur]=s[cur]=1;
        }
        splay(cur);
    }
    void remove(int x)
    {
        int lst=pre_t(x),nxt=suc_t(x);
        splay(lst),splay(nxt,lst);
        int del=ls(nxt);
        if(cnt[del]>1)
        {
            cnt[del]--;
            splay(del);
        }
        else
        {
            q.push(ls(nxt));
            ls(nxt)=0,pushup(nxt),pushup(rt);
        }
    }
    int rnk(int x){find(x);return s[ch[0][rt]];}//x元素的排名
    int kth(int x){return v[kth_t(x+1)];}//排名为第k大的元素
    int pre(int x){return v[pre_t(x)];}
    int suc(int x){return v[suc_t(x)];}
#undef ls
#undef rs
};
```

3 树状数组

```
class BinaryTree
{
#define lowbit(x) (x&(-x))
private:
    int *c,n;
    int query(int x)
    {
        int res=0;
        while(x>0)
        {
            res+=c[x];
            x-=lowbit(x);
        }
        return res;
    }
public:
    BinaryTree(int size)
    {
        n=size;
        c=new int[n+5]();
```

```
    }
    ~BinaryTree()
    {
        delete [] c;
    }
    void modify(int x)
    {
        while(x<=n)
        {
            c[x]++;
            x+=lowbit(x);
        }
    }
    int ask(int l,int r)
    {
        if(l>r) return 0;
        return query(r)-query(l-1);
    }
#undef lowbit
};
```

4 ST表

```
void build()
{
    log[0]=-1;
    for(int i=1;i<=n;i++)
    {
        f[i][0]=a[i-1];
        log[i]=log[i>>1]+1;
    }
    for(int j=1;(1<<j)<=n;j++)
        for(int i=1;i+(1<<j)-1<=n;i++)
            f[i][j]=max(f[i][j-1],f[i+(1<<j-1)][j-1]);
}
int query(int l,int r)
{
    int k=log[r-l+1];
    return max(f[l][k],f[r-(1<<k)+1][k]);
}
```

图论

1 网络流

1.1 最大流

复杂度$\Theta(n^2m)$

```cpp
#include<bits/stdc++.h>
using namespace std;
const int N=1e4+5,maxm=4e5+5,inf=0x3f3f3f3f;
struct edge
{
    int nxt,to,f;
}e[maxm];
int n,m;
int head[N],cur[N],dep[N],tot=1;
void add(int u,int v,int w)
{
    e[++tot]=(edge){head[u],v,w},head[u]=tot;
    e[++tot]=(edge){head[v],u,0},head[v]=tot;
}
bool bfs(int s,int t)
{
    memset(dep,inf,sizeof(dep));
    for(int i=0;i<=n;i++)//or t
        cur[i]=head[i];
    queue<int>q;
    dep[s]=0,q.push(s);
    while(!q.empty())
    {
        int u=q.front();q.pop();
        for(int i=head[u];i!=0;i=e[i].nxt)
        {
            int v=e[i].to,f=e[i].f;
            if(dep[v]!=inf||f<=0) continue;
            dep[v]=dep[u]+1;
            q.push(v);
        }
    }
    return dep[t]!=inf;
}
int dfs(int u,int t,int now)
{
    if(u==t||now==0) return now;
    int flow=0;
    for(int &i=cur[u];i!=0;i=e[i].nxt)
    {
        int v=e[i].to,res=e[i].f;
        if(dep[v]!=dep[u]+1||res<=0) continue;
        int f=dfs(v,t,min(now,res));
        now-=f,flow+=f;
        e[i].f-=f,e[i^1].f+=f;
```

```
        if(now==0) break;
    }
    return flow;
}
int dinic(int s,int t)
{
    int flow=0;
    while(bfs(s,t))
        flow+=dfs(s,t,inf);
    return flow;
}
int main()
{
    int s,t;
    scanf("%d%d%d%d",&n,&m,&s,&t);
    for(int i=1;i<=m;i++)
    {
        int x,y,z;
        scanf("%d%d%d",&x,&y,&z);
        add(x,y,z);
    }
    cout<<dinic(s,t)<<endl;
    return 0;
}
```

1.2 最小费用最大流

```
#include<bits/stdc++.h>
using namespace std;
const int maxn=5e3+5,maxm=1e5+5,inf=0x3f3f3f3f;
int n,m,s,t,maxflow,mincost,tot=1,head[maxn],vis[maxn],dis[maxn];
struct edge
{
    int nxt,to,f,w;
}e[maxm];
void add(int u,int v,int f,int w)
{
    e[++tot]=(edge){head[u],v,f,w},head[u]=tot;
    e[++tot]=(edge){head[v],u,0,-w},head[v]=tot;
}
bool SPFA(int s,int t)
{
    for(int i=0;i<=t;i++)
        vis[i]=0,dis[i]=inf;
    dis[s]=0;vis[s]=1;
    queue<int>q;
    q.push(s);
    while(!q.empty())
    {
        int u=q.front();
        q.pop();
        vis[u]=0;
        for(int i=head[u];i!=0;i=e[i].nxt)
        {
```

```
            int v=e[i].to;
            if(dis[v]>dis[u]+e[i].w&&e[i].f>0)
            {
                dis[v]=dis[u]+e[i].w;
                if(!vis[v])
                {
                    q.push(v);
                    vis[v]=1;
                }
            }
        }
    }
    return dis[t]!=inf;
}
int dfs(int u,int t,int add)
{
    if(u==t||add==0)
    {
        vis[t]=1;
        maxflow+=add;
        return add;
    }
    int flow=0;
    vis[u]=1;
    for(int i=head[u];i!=0;i=e[i].nxt)
    {
        int v=e[i].to,w=e[i].w;
        if((vis[v]==0||v==t)&&e[i].f>0&&dis[v]==dis[u]+w)
        {
            int f=dfs(v,t,min(add,e[i].f));
            mincost+=w*f;
            e[i].f-=f,e[i^1].f+=f;
            flow+=f,add-=f;
            if(add==0) break;
        }
    }
    return flow;
}
void mcmf(int s,int t)
{
    while(SPFA(s,t))
    {
        vis[t]=1;
        while(vis[t])
        {
            memset(vis,0,sizeof(vis));
            dfs(s,t,inf);
        }
    }
}
int main()
{
    scanf("%d%d%d%d",&n,&m,&s,&t);
    while(m--)
    {
```

```
        int u,v,f,w;
        scanf("%d%d%d%d",&u,&v,&f,&w);
        add(u,v,f,w);
    }
    mcmf(s,t);
    printf("%d %d",maxflow,mincost);
    return 0;
}
```

1.3 二分图

性质

最小点覆盖 = 最大匹配

最小边覆盖 = 最大独立集 = 顶点总数 - 最大匹配数

2 Link Cut Tree

```
class LinkCutTree
{
#define ls(k) (ch[0][k])
#define rs(k) (ch[1][k])
private:
    int *ch[2],*s,*f,*r,*v;
    void flip(int x){r[x]^=1;swap(ls(x),rs(x));}
    void pushup(int k){s[k]=s[ls(k)]^s[rs(k)]^v[k];}
    bool chk(int x){return rs(f[x])==x;}
    bool get(int x){return rs(f[x])==x||ls(f[x])==x;}
    void pushdown(int x)
    {
        if(!r[x]) return;
        if(ls(x)) flip(ls(x));
        if(rs(x)) flip(rs(x));
        r[x]=0;
    }
    void pushadd(int x)
    {
        if(get(x)) pushadd(f[x]);
        pushdown(x);
    }
    void rotate(int x)
    {
        int y=f[x],z=f[y],k=chk(x);
        ch[k][y]=ch[k^1][x],f[ch[k^1][x]]=y;
        f[x]=z;if(get(y))ch[chk(y)][z]=x;
        ch[k^1][x]=y,f[y]=x;
        pushup(y),pushup(x);
    }
    void splay(int x)
    {
        pushadd(x);
        while(get(x))
        {
            int y=f[x];
```

```
            if(get(y)) chk(x)!=chk(y)?rotate(x):rotate(y);
            rotate(x);
        }
    }
    void access(int x)
    {
        for(int y=0;x;y=x,x=f[x])
            splay(x),rs(x)=y,pushup(x);
    }
    void makert(int x)
    {
        access(x);
        splay(x);
        flip(x);
    }
    int findrt(int x)
    {
        access(x),splay(x);
        pushdown(x);
        while(ls(x)) x=ls(x),pushdown(x);
        return x;
    }
    void split(int x,int y)
    {
        makert(x);
        access(y);
        splay(y);
    }
public:
    LinkCutTree(int n)
    {
        int size=n+5;
        ch[0]=new int[size]();
        ch[1]=new int[size]();
        s=new int[size]();
        f=new int[size]();
        r=new int[size]();
        v=new int[size]();
    }
    ~LinkCutTree()
    {
        delete [] ch[0];
        delete [] ch[1
        delete [] s;
        delete [] f;
        delete [] r;
        delete [] v;
    }
    void link(int x,int y)
    {
        makert(x);
        if(findrt(y)!=x) f[x]=y;
    }
    void cut(int x,int y)
    {
```

```
        split(x,y);
        if(findrt(y)==x&&!rs(x)&&f[x]==y)
        {
            ls(y)=f[x]=0;
            pushup(y);
        }
    }
    int query(int x,int y)//返回x到y的异或和
    {
        split(x,y);
        return s[y];
    }
    void update(int x,int w)
    {
        splay(x),v[x]=w,pushup(x);
    }
};
```

4 树链剖分

```
#include<bits/stdc++.h>
using namespace std;
const int N=1e5+5;
struct edge
{
    int nxt,to;
}e[N<<1];
int n,m,r,p;
int head[N],tot,seg[N],rev[N],sum[N<<2],tag[N<<2],size[N],top[N],w[N],dep[N],f[N],son[N];
void edge_add(int u,int v)
{
    e[++tot]=(edge){head[u],v},head[u]=tot;
    e[++tot]=(edge){head[v],u},head[v]=tot;
}
void dfs1(int u,int fa)
{
    f[u]=fa;
    size[u]=1;
    dep[u]=dep[fa]+1;
    for(int i=head[u];i!=0;i=e[i].nxt)
    {
        int v=e[i].to;
        if(v==fa) continue;
        dfs1(v,u);
        size[u]+=size[v];
        if(size[son[u]]<size[v]) son[u]=v;
    }
}
void dfs2(int u)
{
    if(son[u])
    {
        seg[son[u]]=++seg[0];
        rev[seg[0]]=son[u];
```

```
        top[son[u]]=top[u];
        dfs2(son[u]);
    }
    for(int i=head[u];i!=0;i=e[i].nxt)
    {
        int v=e[i].to;
        if(top[v]) continue;
        seg[v]=++seg[0];
        rev[seg[0]]=v;
        top[v]=v;
        dfs2(v);
    }
}
void modify(int k,int l,int r,int w)
{
    (tag[k]+=w)%=p;
    (sum[k]+=(r-l+1)*w)%=p;
}
void pushup(int k){sum[k]=sum[k<<1]+sum[k<<1|1];}
void pushdown(int k,int l,int r)
{
    if(!tag[k]) return ;
    int mid=l+r>>1;
    modify(k<<1,l,mid,tag[k]);
    modify(k<<1|1,mid+1,r,tag[k]);
    tag[k]=0;
}
void build(int k,int l,int r)
{
    if(l==r)
    {
        sum[k]=w[rev[l]];
        return;
    }
    int mid=l+r>>1;
    build(k<<1,l,mid);
    build(k<<1|1,mid+1,r);
    pushup(k);
}
int query(int k,int l,int r,int x,int y )
{
    if(l>y||r<x) return 0;
    if(x<=l&&r<=y) return sum[k];
    pushdown(k,l,r);
    int mid=l+r>>1;
    return (query(k<<1,l,mid,x,y)+query(k<<1|1,mid+1,r,x,y))%p;
}
void update(int k,int l,int r,int x,int y,int z)
{
    if(l>y||r<x) return;
    if(x<=l&&r<=y)
    {
        modify(k,l,r,z);
        return;
    }
```

```
    pushdown(k,l,r);
    int mid=l+r>>1;
    update(k<<1,l,mid,x,y,z) ;
    update(k<<1|1,mid+1,r,x,y,z);
    pushup(k);
}
void add(int x,int y,int z)
{
    int fx=top[x],fy=top[y];
    while(fx!=fy)
    {
        if(dep[fx]<dep[fy]) swap(x,y),swap(fx,fy);
        update(1,1,n,seg[fx],seg[x],z);
        x=f[fx],fx=top[x];
    }
    if(dep[x]>dep[y])swap(x,y);
    update(1,1,n,seg[x],seg[y],z);
}
int ask(int x,int y)
{
    int fx=top[x],fy=top[y],res=0;
    while(fx!=fy)
    {
        if(dep[fx]<dep[fy]) swap(x,y),swap(fx,fy);
        (res+=query(1,1,n,seg[fx],seg[x]))%=p;
        x=f[fx],fx=top[x];
    }
    if(dep[x]>dep[y])swap(x,y);
    (res+=query(1,1,n,seg[x],seg[y]))%=p;
    return res;
}
int main()
{
    scanf("%d%d%d%d",&n,&m,&r,&p);
    for(int i=1;i<=n;i++)
        scanf("%d",&w[i]);
    for(int i=1;i<n;i++)
    {
        int u,v;
        scanf("%d%d",&u,&v);
        edge_add(u,v);
    }
    dfs1(r,0);
    seg[r]=seg[0]=1;
    rev[1]=top[r]=r;
    dfs2(r);
    build(1,1,n);
    for(int i=1;i<=m;i++)
    {
        int op;
        scanf("%d",&op);
        if(op==1)//x到y加上z
        {
            int x,y,z;
            scanf("%d%d%d",&x,&y,&z);
```

```
            add(x,y,z);
        }
        else if(op==2)//询问x到y的和
        {
            int x,y;
            scanf("%d%d",&x,&y);
            printf("%d\n",ask(x,y));
        }
        else if(op==3)//以x为根的子树加上z
        {
            int x,z;
            scanf("%d%d",&x,&z);
            update(1,1,n,seg[x],seg[x]+size[x]-1,z);
        }
        else//询问以x为根的子树的和
        {
            int x;
            scanf("%d",&x);
            printf("%d\n",query(1,1,n,seg[x],seg[x]+size[x]-1));
        }
    }
    return 0;
}
```

5 Tarjan

5.1 强连通分量

```
stack<int>s;
vector<int>e[N];
int col[N],num,dfn[N],low[N],dfstime;
void tarjan(int u)
{
    s.push(u);
    dfn[u]=low[u]=++dfstime;
    for(auto v:e[u])
    {
        if(!dfn[v])
        {
            tarjan(v);
            low[u]=min(low[u],low[v]);
        }
        else if(!col[v]) low[u]=min(low[u],dfn[v]);
    }
    if(dfn[u]==low[u])
    {
        col[u]=++num;
        while(s.top()!=u)
        {
            col[s.top()]=num;
            s.pop();
        }
        s.pop();
    }
```

```
}
```

5.2 割点（边）

```
if((u==root&&cnt>1)||(u!=root&&dfn[u]<=low[v])) cut[u]=1;
```

6 最短路

6.1 SPFA

```
void SPFA(int s,int t)
{
    queue<int>q;
    bitset<N>vis;
    dis[s]=0,vis[s]=1;
    q.push(s);
    while(!q.empty())
    {
        int u=q.front();
        q.pop();
        vis[u]=0;
        for(int i=head[u];i!=0;i=e[i].nxt)
        {
            int v=e[i].to;
            if(dis[v]>dis[u]+1)
            {
                dis[v]=dis[u]+1;
                if(!vis[v])
                {
                    q.push(v);
                    vis[v]=1;
                }
            }
        }
    }
}
```

6.2 Floyd

```
memset(dis,inf,sizeof(dis));
for(int k=1;k<=n;k++)
    for(int i=1;i<=n;i++)
        for(int j=1;j<=n;j++)
            dis[i][j]=min(dis[i][k]+dis[k][j],dis[i][j]);
```

数论

1 基础知识

1.1 积性函数

对于一个数论函数f，当$n \perp m$时有$f(nm)=f(n)f(m)$，则称其为积性函数

常见的积性函数σ_0、φ，其中，$\sigma_0(p^k)=k+1$，$\varphi(p^k)=p^{k-1}(p-1)$

$$\varphi(n)=\sum_{d\le n}[d\perp n]\Leftrightarrow\varphi(p^k)=p^{k-1}(p-1)$$

$$\sigma_x(n)=\sum_{d|n}d^x\Leftrightarrow\sigma_x(p^k)=\sum_{i=0}^{k}p^{ix}$$

$$\mu(n)=\begin{cases}(-1)^t & n=p_1p_2\ldots p_t\\ 0 & \text{else}\end{cases}$$

重要结论:$n=\sum_{d|n}\varphi(d)$

线性筛积性函数

```
vector<int>p;
bitset<N>vis;
int phi[N],sig[N],miu[N],e[N];
void init()
{
    phi[1]=miu[1]=sig[1]=1;
    for(int i=2;i<N;i++)
    {
        if(!vis[i])
        {
            p.push_back(i);
            phi[i]=i-1;
            miu[i]=-1;
            sig[i]=2;
            e[i]=2;
        }
        for(auto j:p)
        {
            if(i*j>=N) break;
            vis[i*j]=1;
            if(i%j)
            {
                e[i*j]=2;
                phi[i*j]=phi[i]*phi[j];
                miu[i*j]=miu[i]*miu[j];
                sig[i*j]=sig[i]*sig[j];
            }
            else
            {
                e[i*j]=e[i]+1;
```

```
                phi[i*j]=phi[i]*j;
                miu[i*j]=0;
                sig[i*j]=sig[i]/e[i]*(e[i]+1);
                break;
            }
        }
    }
}
```

2 莫比乌斯反演

2.1 狄利克雷卷积

$$h(n)=\sum_{d|n}f(d)g(\frac{n}{d})$$

(1) 满足**交换律、结合律、分配律**

(2) 单位元$\epsilon(n)=[n=1]$

(3) 当$f*g=\epsilon$时，称g是f的逆元。以下考虑一个递归定义：

$$g(n)=\frac{1}{f(1)}\left([n=1]-\sum_{d|n,d\neq 1}f(d)g(\frac{n}{d})\right)$$

2.2 莫比乌斯反演

$$f(n)=\sum_{d|n}g(d)\Leftrightarrow g(n)=\sum_{d|n}\mu(d)f(\frac{n}{d})$$

例题

> 设$d(x)$为x的约数个数，给定n，m，求
>
> $$\sum_{i=1}^{n}\sum_{j=1}^{m}d(ij)$$

解法

$$= \sum_{i=1}^{n}\sum_{j=1}^{m}\sum_{x|i}\sum_{y|j}[gcd(x,y)=1]$$

$$= \sum_{x=1}^{n}\sum_{y=1}^{m}\lfloor\frac{n}{x}\rfloor\lfloor\frac{m}{y}\rfloor[gcd(x,y)=1]$$

$$= \sum_{x=1}^{n}\sum_{y=1}^{m}\lfloor\frac{n}{x}\rfloor\lfloor\frac{m}{y}\rfloor\sum_{d|x,d|y}\mu(d)$$

$$= \sum_{d=1}^{min(n,m)}\mu(d)\sum_{x=1}^{n}\lfloor\frac{n}{x}\rfloor[d|x]\sum_{y=1}^{m}\lfloor\frac{m}{y}\rfloor[d|y]$$

$$= \sum_{d=1}^{min(n,m)}\mu(d)\sum_{x=1}^{\lfloor\frac{n}{d}\rfloor}\lfloor\frac{n}{dx}\rfloor\sum_{x=1}^{\lfloor\frac{m}{d}\rfloor}\lfloor\frac{m}{dy}\rfloor$$

$$= \sum_{d=1}^{min(n,m)}\mu(d)\sum_{x=1}^{\lfloor\frac{n}{d}\rfloor}\lfloor\frac{\lfloor\frac{n}{d}\rfloor}{x}\rfloor\sum_{y=1}^{\lfloor\frac{m}{d}\rfloor}\lfloor\frac{\lfloor\frac{m}{d}\rfloor}{y}\rfloor$$

设

$$f(i)=\sum_{j=1}^{i}\lfloor\frac{i}{j}\rfloor$$

则

$$= \sum_{d=1}^{min(n,m)}\mu(d)f(\lfloor\frac{n}{d}\rfloor)f(\lfloor\frac{m}{d}\rfloor)$$

代码

```
#include<bits/stdc++.h>
using namespace std;
const int maxn=5e4+5;
bool v[maxn];
int p[maxn],cnt,mu[maxn],sum[maxn];
long long g[maxn];
void init(int n)
{
    mu[1]=1;
    for(int i=2;i<=n;i++)
    {
        if(!v[i]){p[++cnt]=i;mu[i]=-1;}
        for(int j=1;j<=cnt&&p[j]*i<=n;j++)
        {
            v[p[j]*i]=1;
            if(i%p[j]==0)break;
            else mu[i*p[j]]=-mu[i];
        }
    }
```

```
    for(int i=1;i<=n;i++)
        sum[i]=sum[i-1]+mu[i];
    for(int i=1;i<=n;i++)
    {
        long long res=0;
        for(int l=1,r;l<=i;l=r+1)
        {
            r=(i/(i/l));
            res+=1ll*(r-l+1)*1ll*(i/l);
        }
        g[i]=res;
    }
}
int main()
{
    int t;
    scanf("%d",&t);
    init(50000);
    while(t--)
    {
        int n,m;
        scanf("%d%d",&n,&m);
        long long res=0;
        for(int l=1,r;l<=min(n,m);l=r+1)
        {
            r=min(n/(n/l),m/(m/l));
            res+=(sum[r]-sum[l-1])*1ll*g[n/l]*1ll*g[m/l];
        }
        printf("%lld\n",res);
    }
    return 0;
}
```

2.3 杜教筛

引理

$$\lfloor\frac{n}{t}\rfloor \geq \lfloor\frac{n}{i}\rfloor \Leftrightarrow \frac{n}{t} \geq \lfloor\frac{n}{i}\rfloor \Leftrightarrow t \leq \lfloor\frac{n}{\lfloor\frac{n}{i}\rfloor}\rfloor$$

筛法 考虑当n极大时如何求$s(n)=\sum_{i=1}^{n} f(i)$

构造

$$\sum_{i=1}^{n}(f*g)(i)=\sum_{i=1}^{n}\sum_{xy=i}f(x)g(y)$$

$$=\sum_{y=1}^{n}g(y)\sum_{xy\leq n}(x)=\sum_{y=1}^{n}g(y)s\left(\lfloor\frac{n}{y}\rfloor\right)$$

移项

$$g(1)s(n)=\sum_{i=1}^{n}(f*g)(i)-\sum_{y=1}^{n}g(y)s\left(\lfloor\frac{n}{y}\rfloor\right)$$

如果能快速计算$\sum_{i=1}^{n}(f*g)(i)$和$\sum_{y=1}^{n}g(y)$的前缀和，那么就能快速求出$s(n)$

根据**引理**对于每一个$s(x)$,都能在$\Theta(\sqrt{x}\ )$内求出

时间复杂度

不难证明当$x\le n^{\frac{2}{3}}$时直接求出所有的$s(x)$，否则递归计算所有的$s(x)$

代码 求S_{φ}和S_{μ}，复杂度$\Theta(n^{\frac{2}{3}})$

```
#include<bits/stdc++.h>
using namespace std;
typedef long long LL;
const int maxn=4000000;
int p[maxn+10],N;
bool phiv[maxn+10],muv[maxn+10],v[maxn+10];
LL mu[maxn+10],phi[maxn+10],ansmu[maxn],ansphi[maxn];
inline void init()
{
    v[1]=mu[1]=phi[1]=1;
    int tot=0;
    for(int i=2;i<=maxn;i++)
    {
        if (!v[i]) p[++tot]=i,mu[i]=-1,phi[i]=i-1;
        for(int j=1;j<=tot&&i*p[j]<=maxn;++j)
        {
            v[i*p[j]]=1;
            if(i%p[j]) mu[i*p[j]]=-mu[i],phi[i*p[j]]=phi[i]*phi[p[j]];
            else {mu[i*p[j]]=0,phi[i*p[j]]=phi[i]*p[j];break;}
        }
    }
    for(int i=1;i<=maxn;++i)
        mu[i]+=mu[i-1],phi[i]+=phi[i-1];
}
LL Sphi(int n)
{
    if(n<=maxn) return phi[n];
    int x=N/n;//利用引理
    if(phiv[x]) return ansphi[x];
    phiv[x]=true;
    LL ans=0;
    for(int l=2,r;l<=n;l=r+1)
        r=n/(n/l),ans+=(r-l+1)*Sphi(n/l);
    return ansphi[x]=n*(n+1ll)/2ll-ans;
}
LL Smu(int n)
{
    if(n<=maxn) return mu[n];
    int x=N/n;
    if(muv[x]) return ansmu[x];
    muv[x]=true;
    LL ans=1;
```

```
    for(int l=2,r;l<=n;l=r+1)
        r=n/(n/l),ans-=(r-l+1)*Smu(n/l);
    return ansmu[x]=ans;
}
int main()
{
    init();
    int t;
    scanf("%d",&t);
    while(t--)
    {
        scanf("%d",&N);
        memset(phiv,0,sizeof(phiv));
        memset(muv,0,sizeof(muv));
        printf("%lld %lld\n",Sphi(N),Smu(N));
    }
    return 0;
}
```

3 快速傅立叶变换

3.1 单位复数根

定义 $\omega_n = e^{2\pi/n}$

折半引理 n个n次单位复数根的平方的集合就是$\frac{n}{2}$个$\frac{n}{2}$次单位复数根的集合

3.2 FFT

DFT

$$y_k = \sum_{i=0}^{n-1} a_i (\omega_n^k)^i$$

根据折半引理

$$y_k = y_k^{[0]} + \omega_n^k y_k^{[1]}$$
$$y_{k+n/2} = y_k^{[0]} - \omega_n^k y_k^{[1]}$$

DFT^{-1}

$$a_k = \frac{1}{n} \sum_{i=0}^{n-1} y_i (\omega_n^{-k})^i$$

模版

```
class FFT
{
private:
```

```
    static constexpr double pi=3.14159265358979323846264338627 9;
    vector<int>rev;
    struct cpx
    {
        double x,y;
        cpx (double a=0,double b=0){x=a,y=b;}
        cpx operator+(const cpx &t) const{return cpx(x+t.x,y+t.y);}
        cpx operator-(const cpx &t) const{return cpx(x-t.x,y-t.y);}
        cpx operator*(const cpx &t) const{return cpx(x*t.x-y*t.y,x*t.y+y*t.x);}
    };
    void DFT(vector<cpx>&a,int lim,int type)
    {
        for(int i=0;i<lim;i++)
            if(i<rev[i]) swap(a[i],a[rev[i]]);
        for(int i=1;i<lim;i<<=1)//半区间长度
        {
            cpx wn(cos(pi/i),type*sin(pi/i));
            for(int j=0;j<lim;j+=(i<<1))//当前处理点
            {
                cpx w(1,0);
                for(int k=0;k<i;k++,w=w*wn)
                {
                    cpx x=a[j+k],y=w*a[j+k+i];
                    a[j+k]=x+y,a[j+k+i]=x-y;
                }
            }
        }
    }
public:
    vector<int>cal(vector<int>t1,vector<int>t2)
    {//调用cal传入两个包含多项式系数的vector，返回卷积
        int n=t1.size(),m=t2.size(),lim=1,l=0,len=n+m-1;
        while(lim<n+m-1) lim<<=1,l++;
        vector<cpx>a(lim),b(lim);
        for(int i=0;i<n;i++) a[i].x=t1[i];
        for(int i=0;i<m;i++) b[i].x=t2[i];
        rev.resize(lim);
        for(int i=0;i<lim;i++)
            rev[i]=(rev[i>>1]>>1)|((i&1)<<(l-1));//逆二进制
        DFT(a,lim,1),DFT(b,lim,1);
        for(int i=0;i<lim;i++)
            a[i]=a[i]*b[i];
        DFT(a,lim,-1);
        vector<int>c;
        for(int i=0;i<len;i++)
            c.push_back((int)(a[i].x/(double)lim+0.5));
        return c;
    }
};
```

3.3 NTT

在取模意义下$g_n = g^{(p-1)/n}$，等价于$\omega_n = e^{2\pi i/n}$

通常$g = 3, p = 998244353$

4 排列组合

4.1 线性筛法

```
int inv[N],fac[N];
int C(int n,int m){return fac[n]*inv[n-m]%mod*inv[m]%mod;}
int A(int n,int m){return fac[n]*inv[n-m]%mod;}
void init()
{
    inv[0]=fac[0]=inv[1]=1;
    for(int i=1;i<N;i++) fac[i]=fac[i-1]*i%mod;
    for(int i=2;i<N;i++) inv[i]=(mod-mod/i)*inv[mod%i]%mod;
    for(int i=1;i<N;i++) inv[i]=inv[i-1]*inv[i]%mod;
}
```

计算几何

1 基础知识

1.1 常用量

```
const double eps=1e-8;
const double inf=1e20;
const double pi=acos(-1.0);
int sgn(double x)
{
    if(fabs(x)<eps)return 0;
    if(x<0)return -1;
    else return 1;
}
inline double sqr(double x){return x*x;}
```

1.2 点

```
struct point
{
    double x,y;
    point(){x=0,y=0;}
    point(double _x,double _y){x=_x,y=_y;}
    void input(){scanf("%lf%lf",&x,&y);}
    void output(){printf("%.2f %.2f\n",x,y);}
    bool operator==(point b)const{return sgn(x-b.x)==0&&sgn(y-b.y)==0;}
    bool operator<(point b)const{return sgn(x-b.x)==0?sgn(y-b.y)<0:x<b.x;}
    point operator+(const point &b)const{return point(x+b.x,y+b.y);}
    point operator-(const point &b)const{return point(x-b.x,y-b.y);}
    point operator*(const double &k)const{return point(x*k,y*k);}
    point operator/(const double &k)const{return point(x/k,y/k);}
    double operator^(const point &b)const{return x*b.y-y*b.x;}//叉积
    double operator*(const point &b)const{return x*b.x+y*b.y;}//点积
    double len(){return hypot(x,y);}
    double distance(point p){return hypot(x-p.x,y-p.y);}
    double rad(point a,point b)//计算pa,pb形成的夹角
    {
        point p=*this;
        return fabs(atan2(fabs((a-p)^(b-p)),(a-p)*(b-p)));
    }
    point trunc(double r)//转化长度为r的向量
    {
        double l=len();
        if(!sgn(l))return *this;
        r/=l;
        return point(x*r,y*r);
    }
    point rotleft(){return point(-y,x);}
    point rotright(){return point(y,-x);}
    point rotate(point p,double angle)
    {
```

```
        point v=(*this)-p;
        double c=cos(angle),s=sin(angle);
        return point(p.x+v.x*c-v.y*s,p.y+v.x*s+v.y*c);
    }
};
```

1.3 线

```
struct line
{
    point s,e;
    line(){};
    line(point _s,point _e){s=_s,e=_e;}
    bool operator==(const line &l){return l.s==s&&l.e==e;}
    line(point p,double angle)//点和角度
    {
        s=p;
        if(!sgn(angle-pi/2)) e=(s+point(0,1));
        else e=(s+point(1,tan(angle)));
    }
    line(double a,double b,double c)//ax+by+c=0
    {
        if(sgn(a)==0) s=point(0,-c/b),e=point(1,-c/b);
        else if(sgn(b)==0) s=point(-c/a,0),e=point(-c/a,1);
        else s=point(0,-c/b),e=point(1,(-c-a)/b);
    }
    void input(){s.input(),e.input();}
    void adjust(){if(e<s)swap(s,e);}
    double length(){return s.distance(e);}
    double angle()//返回直线倾斜角 0<=theta<pi`
    {
        double k=atan2(e.y-s.y,e.x-s.x);
        if(sgn(k)<0) k+=pi;
        if(sgn(k-pi)==0) k-=pi;
        return k;
    }
    int relation(point p)//点和直线关系 0.在左侧 1.在右侧 2.在直线上
    {
        int c=sgn((p-s)^(e-s));
        if(c<0) return 0;
        else if(c>0) return 1;
        else return 2;
    }
    bool pointonseg(point p){return sgn((p-s)^(e-s))==0&&sgn((p-s)*(p-e))<=0;}
    bool parallel(line l){return sgn((e-s)^(l.e-l.s))==0;}
    int segcrossseg(line l)//0.不相交 1.非规范相交 2.规范相交
    {
        int d1=sgn((e-s)^(l.s-s));
        int d2=sgn((e-s)^(l.e-s));
        int d3=sgn((l.e-l.s)^(s-l.s));
        int d4=sgn((l.e-l.s)^(e-l.s));
        if( (d1^d2)==-2&&(d3^d4)==-2 )return 2;
        return (d1==0&&sgn((l.s-s)*(l.s-e))<=0) ||
            (d2==0&&sgn((l.e-s)*(l.e-e))<=0) ||
```

```
            (d3==0&&sgn((s-l.s)*(s-l.e))<=0) ||
            (d4==0&&sgn((e-l.s)*(e-l.e))<=0);
    }
    int linecrossseg(line l)//0.不相交 1.非规范相交 2.规范相交
    {
        int d1=sgn((e-s)^(l.s-s));
        int d2=sgn((e-s)^(l.e-s));
        if((d1^d2)==-2) return 2;
        return (d1==0||d2==0);
    }
    //0.平行 1.重合 2.相交
    int linecrossline(line l){return (*this).parallel(l)?l.relation(s)==3:2;}
    point crosspoint(line l)//必须要有交点才可以求
    {
        double a1=(l.e-l.s)^(s-l.s);
        double a2=(l.e-l.s)^(e-l.s);
        return point((s.x*a2-e.x*a1)/(a2-a1),(s.y*a2-e.y*a1)/(a2-a1));
    }
    double dispointtoline(point p){return fabs((p-s)^(e-s))/length();}
    double dispointtoseg(point p)//点到线段的距离
    {
        if(sgn((p-s)*(e-s))<0||sgn((p-e)*(s-e))<0)
            return min(p.distance(s),p.distance(e));
        return dispointtoline(p);
    }
    double dissegtoseg(line l)
    {
        double res1=min(dispointtoseg(l.s),dispointtoseg(l.e));
        double res2=min(l.dispointtoseg(s),l.dispointtoseg(e));
        return min(res1,res2);
    }
    //返回点p在直线上的投影
    point lineprog(point p){return s+( ((e-s)*((e-s)*(p-s)))/((e-s).len()) );}
    point symmetrypoint(point p)//返回点p关于直线的对称点
    {
        point q=lineprog(p);
        return point(2*q.x-p.x,2*q.y-p.y);
    }
};
```

其它算法

1 莫队

1.1普通莫队

离线询问后将答案排序，`block`=$n/\sqrt{m}$，复杂度$\Theta(n\sqrt{m})$

```
friend bool operator<(const node a,const node b)
{
    if((a.l/block)==(b.l/block)) return ((a.l/block%2)?a.r>b.r:a.r<b.r);
    else return a.l<b.l;
}
```

1.2带修莫队

加一维时间，`block`=$n^{\frac{2}{3}}$，复杂度$\Theta(n^{\frac{5}{3}})$

```
friend bool operator<(const node a,const node b)
{
    if(a.l/block!=b.l/block) a.l/block<b.l/block;
    if(a.r/block!=b.r/block) a.r/block<b.r/block;
    return a.k<b.k;
}
```

2 分块

2.1 树套树

```
#include<bits/stdc++.h>
using namespace std;
const int N=5e4+5,inf=0x7fffffff;
int a[N],b[N],num,n,m,blg[N],L[N],R[N];
inline int read()
{
    int x=0,f=1;
    char c=getchar();
    while(!isdigit(c)){if(c=='-') f=false;c=getchar();}
    while(isdigit(c)){x=(x<<3)+(x<<1)+(c-'0');c=getchar();}
    return f?x:-x;
}
void build()
{
    int t=sqrt(n);
    num=n/t;
    if(n%t) num++;
    for(int i=1;i<=n;i++)
        blg[i]=(i-1)/t+1,b[i]=a[i];
    for(int i=1;i<=num;i++)
        L[i]=(i-1)*t+1,R[i]=i*t;
    R[num]=n;
```

```
    for(int i=1;i<=num;i++)
        sort(b+L[i],b+R[i]+1);
}
int search(int l,int r,int k,int type)
{
    int p=0;
    if(!type)
    {
        while(l<=r)
        {
            int mid=(l+r)>>1;
            if(b[mid]<k) p=mid,l=mid+1;
            else r=mid-1;
        }
    }
    else
    {
        while(l<=r)
        {
            int mid=(l+r)>>1;
            if(b[mid]>k) p=mid,r=mid-1;
            else l=mid+1;
        }
    }
    return p;
}
int find(int k,int r,int l)
{
    int cnt=1;
    if(blg[l]==blg[r])
    {
        for(int i=l;i<=r;i++)
            if(a[i]<k) cnt++;
        return cnt;
    }
    for(int i=l;i<=R[blg[l]];i++) if(a[i]<k) cnt++;
    for(int i=L[blg[r]];i<=r;i++) if(a[i]<k) cnt++;
    for(int i=blg[l]+1;i<blg[r];i++)
        if(b[L[i]]<k) cnt+=search(L[i],R[i],k,0)-(L[i]-1);
    return cnt;
}
int order(int k,int r,int l)
{
    if(blg[l]==blg[r])
    {
        vector<int>v;
        for(int i=l;i<=r;i++)
            v.push_back(a[i]);
        sort(v.begin(),v.end());
        return v[k-1];
    }
    int lk=0,rk=1e8,res;
    while(lk<=rk)
    {
        int mid=(lk+rk)>>1;
```

```
        if(find(mid,r,l)<=k) res=mid,lk=mid+1;
        else rk=mid-1;
    }
    return res;
}
void change(int k,int p)
{
    a[p]=k;
    for(int i=L[blg[p]];i<=R[blg[p]];i++)
        b[i]=a[i];
    sort(b+L[blg[p]],b+R[blg[p]]+1);
}
int pre(int k,int r,int l)
{
    #define turn(i,a) (a[i]<k&&a[i]>res)?res=a[i]:res=res
    int res=-inf;
    if(blg[l]==blg[r])
    {
        for(int i=l;i<=r;i++) turn(i,a);
        return res==inf?-res:res;
    }
    for(int i=l;i<=R[blg[l]];i++) turn(i,a);
    for(int i=L[blg[r]];i<=r;i++) turn(i,a);
    for(int i=blg[l]+1;i<blg[r];i++)
        if(b[L[i]]<k) turn(search(L[i],R[i],k,0),b);
    return res;
    #undef turn
}
int suc(int k,int r,int l)
{
    #define turn(i,a) (a[i]>k&&a[i]<res)?res=a[i]:res=res
    int res=inf;
    if(blg[l]==blg[r])
    {
        for(int i=l;i<=r;i++) turn(i,a);
        return res==-inf?-res:res;
    }
    for(int i=l;i<=R[blg[l]];i++) turn(i,a);
    for(int i=L[blg[r]];i<=r;i++) turn(i,a);
    for(int i=blg[l]+1;i<blg[r];i++)
        if(b[R[i]]>k) turn(search(L[i],R[i],k,1),b);
    return res;
    #undef turn
}
int main()
{
    n=read(),m=read();
    for(int i=1;i<=n;i++)
        a[i]=read();
    build();
    while(m--)
    {
        int op=read();
        if(op==1) printf("%d\n",find(read(),read(),read()));//查询k在区间内的排名
        if(op==2) printf("%d\n",order(read(),read(),read()));//查询区间内排名为k的值
```

```
        if(op==3) change(read(),read());//修改某一位值上的数值
        if(op==4) printf("%d\n",pre(read(),read(),read()));//查询k在区间内的前驱
        if(op==5) printf("%d\n",suc(read(),read(),read()));//查询k在区间内的后驱
    }
    return 0;
}
```